Java Web Services - Individuell Laboration: Reflektion

I denna laboration utvecklade jag ett säkert REST API för ett bloggsystem där användare kan skapa, läsa, uppdatera och radera blogginlägg. Säkerheten hanterades med Keycloak, och målet var att skydda alla endpoints baserat på användarens roll och identitet.

Säkerhet

För att implementera säkerhet skapade jag en ny realm i Keycloak med en klient och flera användare med rollerna user och admin. Jag kopplade sedan ihop Keycloak med Spring Boot via inställningarna i application.properties.

Jag valde att använda SecurityFilterChain för att hantera rollbaserad åtkomst, eftersom det gav mig mer flexibilitet jämfört med @PreAuthorize och fungerade bättre för att konfigurera hela säkerhetsflödet på ett ställe. Jag fick justera så att rollerna stämde exakt med dem i Keycloak, vilket var en viktig del för att autentiseringen skulle fungera korrekt.

För att säkerställa att endast rätt användare kunde uppdatera eller radera sina blogginlägg implementerade jag en kontroll som jämför användarens sub (Keycloak-ID) från access token med authorId på blogginlägget. Vid skapandet av ett nytt blogginlägg hämtas sub från token och sparas som authorId, vilket gör det möjligt att koppla varje inlägg till rätt användare.

Design

Jag strukturerade applikationen enligt en typisk lagerarkitektur i Spring Boot: **Controller-lagret** hanterar HTTP-endpoints, **Service-lagret** innehåller affärslogiken, **Repository-lagret** ansvarar för databasåtkomst.

Genom att isolera logiken i service-lagret kunde jag återanvända funktionalitet på ett tydligare sätt och hålla controller-klasserna mer fokuserade och lättlästa. Jag använde också ResponseEntity för att returnera HTTP-svar på ett konsekvent sätt, samt implementerade en egen ResourceNotFoundException för att hantera fall där ett blogginlägg inte hittades.

Datamodellen var enkel – en BlogPost-entitet med fält för title, content och authorId. Jag använde en in-memory H2-databas och laddade in några exempelinlägg med kända användar-ID:n via data.sql för att kunna testa funktionaliteten.

Övriga reflektioner

En av de största utmaningarna var att hitta ett tillförlitligt sätt att verifiera att rätt användare raderade ett blogginlägg. Detta krävde noggrann testning med Postman och kontroll av access tokens. Jag lärde mig mycket om hur JWT-tokens fungerar och hur man validerar dem med verktyg som jwt.io.

Överlag gav projektet mig en djupare förståelse för autentisering och auktorisering med Keycloak, samt hur man bygger ett säkert och väldesignat API i Java med Spring Boot.

Detta projekt kombinerade backend-utveckling med verkliga säkerhetsprinciper, vilket gjorde det både tekniskt utmanande och givande. Jag har nu en mycket tydligare förståelse för hur man bygger säkra API:er med Spring Boot och Keycloak.